

Software Engineering & System Architecture

การออกแบบระดับคอมโพเนนท์ (Modeling Component-Level Design)

รายวิชาการวิเคราะห์และออกแบบระบบซอฟต์แวร์

สมาชิกในกลุ่มผู้จัดทำ



1 อรปรียา อาสนทอง 6605001699

2 อภิญญา วงษ์ศรี 6605005344

3 กิมทีนา ลี 6705006069

4 สุธิดา ทรงรัมย์ 6605005716

5 วิญญู สายแสง 6805007405



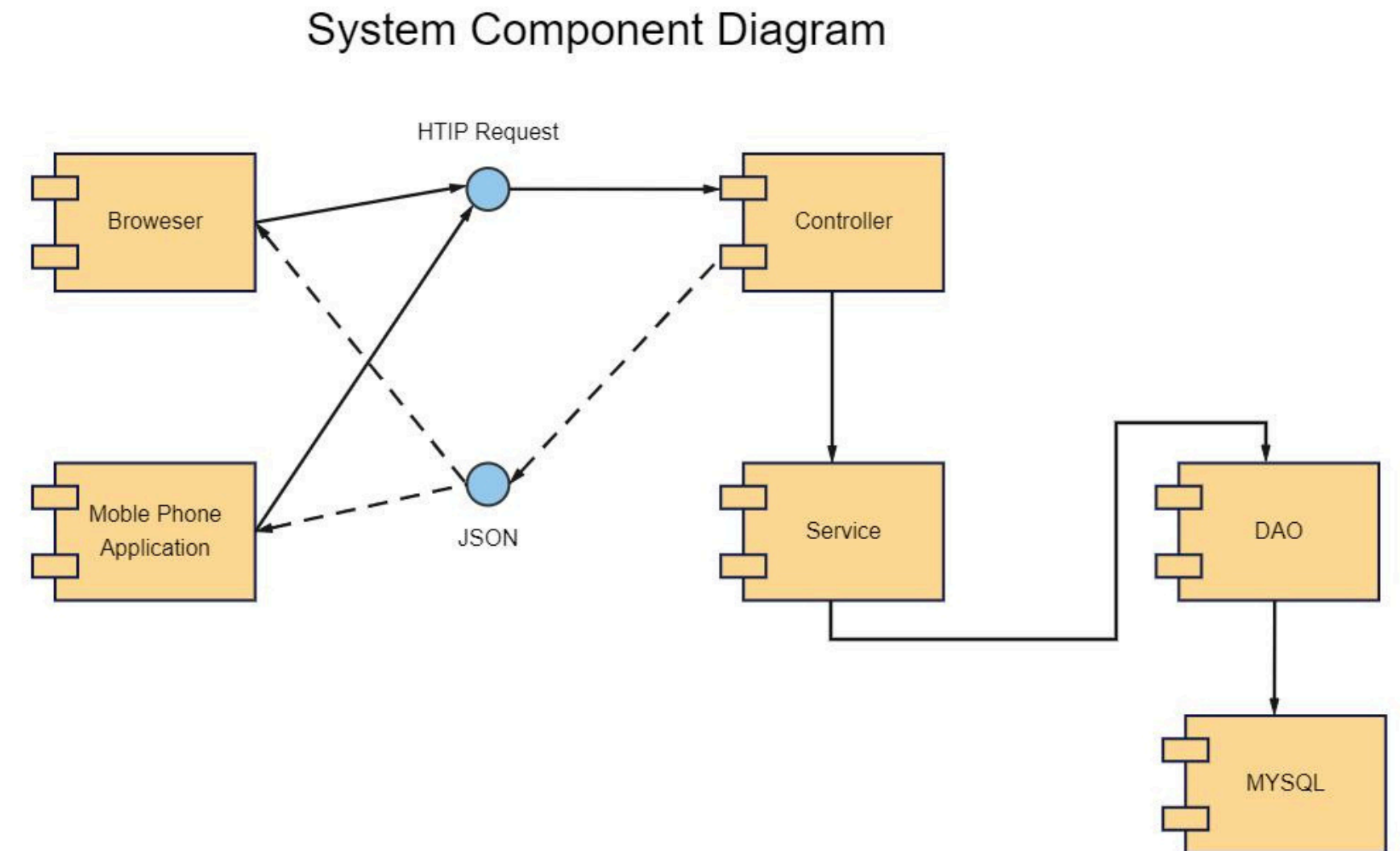
1.1 ความหมายของคอมโพเนนต์ (Component)

📌 นิยามของ Component

ส่วนประกอบหนึ่งของซอฟต์แวร์ที่ถูกออกแบบให้มีหน้าที่หรือความรับผิดชอบเฉพาะ และสามารถทำงานร่วมกับส่วนประกอบอื่นผ่านอินเทอร์เฟซ (Interface) ที่กำหนดไว้

🏗️ รูปแบบของคอมโพเนนต์

โมดูล (Module) | คลาส (Class) | กลุ่มของคลาส | ฟังก์ชันหรือชุดฟังก์ชัน | ไลบรารี (Library) | บริการ (Service)



แนวคิดสำคัญ: การแบ่งระบบขนาดใหญ่ออกเป็นส่วนย่อยที่จัดการได้ง่าย

มีหน้าที่ชัดเจนและติดต่อผ่านช่องทางที่กำหนด

ตัวอย่าง: ระบบจองคิวร้านตัดผม แบ่งเป็น Booking, Queue, Payment และ Notification

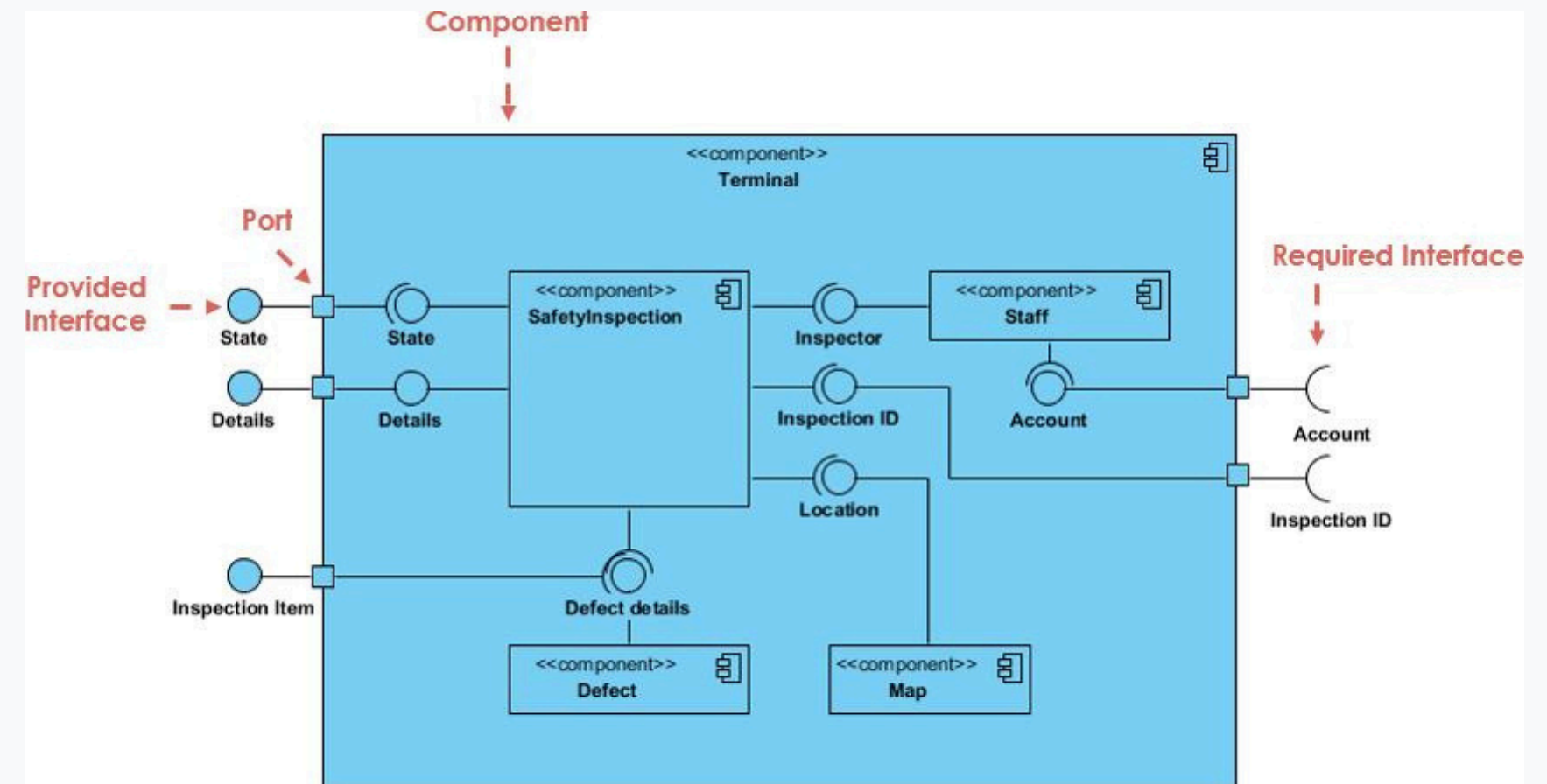
1.2 การออกแบบระดับคอมโพเนนต์ (Component-Level Design)

กระบวนการออกแบบรายละเอียดภายในของคอมโพเนนต์แต่ละส่วนของระบบ รวมถึงกำหนดหน้าที่ ข้อมูล การทำงาน อินเทอร์เฟซ และความสัมพันธ์กับคอมโพเนนต์อื่น

*เป็นขั้นตอนที่นำแบบจำลองระดับสถาปัตยกรรมมาแตกเป็นรายละเอียดเพื่อเขียนโปรแกรมจริง

≡ สิ่งที่ต้องกำหนดในการออกแบบ

- ✓ 1. หน้าที่และความรับผิดชอบของคอมโพเนนต์
- ✓ 2. ข้อมูลที่คอมโพเนนต์ต้องใช้และจัดเก็บ
- ✓ 3. กระบวนการหรืออัลกอริทึมภายใน
- ✓ 4. อินเทอร์เฟซสำหรับติดต่อกับคอมโพเนนต์อื่น
- ✓ 5. ความสัมพันธ์และการพึ่งพาระหว่างคอมโพเนนต์



เป้าหมายหลัก: ทำให้ซอฟต์แวร์มีโครงสร้างที่เข้าใจง่าย แก้ไขและทดสอบได้ง่าย และสามารถนำส่วนประกอบกลับมาใช้ซ้ำได้ (Reusable)

1.3 การออกแบบซอฟต์แวร์เชิงโครงสร้าง (Structured Software Design)

มองระบบเป็นกลุ่มของกระบวนการหรือฟังก์ชัน และแบ่งระบบออกเป็นโมดูลย่อยตามหน้าที่ โดยใช้หลักการ Decomposition (การแบ่งปัญหาเป็นส่วนย่อย)

★ ลักษณะสำคัญ

- ✓ เน้น ฟังก์ชันหรือกระบวนการ (Function/Process) เป็นหลัก
- ✓ แบ่งระบบออกเป็นโมดูลย่อยเรียงลำดับชั้น
- ✓ เน้นการควบคุมการไหลของข้อมูลและการประมวลผล
- ✓ พยายามให้แต่ละโมดูลมี Cohesion สูง และ Coupling ต่ำ

ตัวอย่าง: ระบบสั่งซื้อสินค้า

ระบบสั่งซื้อสินค้า

- รับข้อมูลลูกค้า
- จัดการสินค้า
- คำนวณราคา
- บันทึกคำสั่งซื้อ
- ออกใบเสร็จ

1.3 Structured Design: ข้อดี และ ข้อจำกัด

ข้อดี (Pros)

- ✓ เหมาะกับระบบที่สามารถแยกเป็นกระบวนการชัดเจน
- ✓ ช่วยให้การวิเคราะห์ลำดับการทำงาน (Process Workflow) ทำได้ง่ายและเป็นขั้นตอน
- ✓ โครงสร้างการทำงานเข้าใจได้ง่ายไม่ซับซ้อนในระบบขนาดเล็กถึงปานกลาง

ข้อจำกัด (Limitations)

- ✓ เมื่อระบบมีขนาดใหญ่และมีข้อมูลหรือสถานะที่ซับซ้อน การจัดการอาจทำได้ยาก
- ✓ การซ่อนหรือปกป้องข้อมูลทำได้ยากกว่าแนวทางเชิงวัตถุ
- ✓ ความสัมพันธ์ระหว่างข้อมูลและฟังก์ชันแยกจากกัน ทำให้แก้ไขได้ลำบากเมื่อความต้องการเปลี่ยน

1.4 การออกแบบซอฟต์แวร์เชิงวัตถุ (Object-Oriented Design)

มองระบบเป็นกลุ่มของ วัตถุ (Objects) ซึ่งรวมข้อมูล (Attributes) และพฤติกรรม (Methods) ไว้ด้วยกัน โดยสร้างขึ้นมาจาก คลาส (Class)

Encapsulation:

รวมข้อมูลและพฤติกรรมไว้ในหน่วยเดียว

Abstraction:

แสดงเฉพาะส่วนสำคัญ ซ่อนรายละเอียด

Inheritance:

การสืบทอดคุณสมบัติจากคลาสอื่น

Polymorphism:

การตอบสนองที่หลากหลายในรูปแบบเดียวกัน

Class: Customer

- customer_id
- name
- + bookService()

Class: Booking

- booking_id, date, status
- + createBooking()
- + cancelBooking()

Class: Payment

- payment_id, amount
- + processPayment()

ข้อดี: เหมาะกับระบบขนาดใหญ่ที่มีความซับซ้อน ช่วยให้นำคลาสกลับมาใช้ซ้ำ (Reuse) และดูแลรักษาระบบง่าย

U 1.5 Cohesion (ความเป็นเอกภาพของโมดูล)

© นิยาม Cohesion

ระดับที่องค์ประกอบภายในคอมโพเนนต์หรือโมดูลมีความเกี่ยวข้องและมุ่งไปสู่หน้าที่เดียวกันมากน้อยเพียงใด

หลักการออกแบบที่ดี: High Cohesion (ความเป็นเอกภาพสูง)

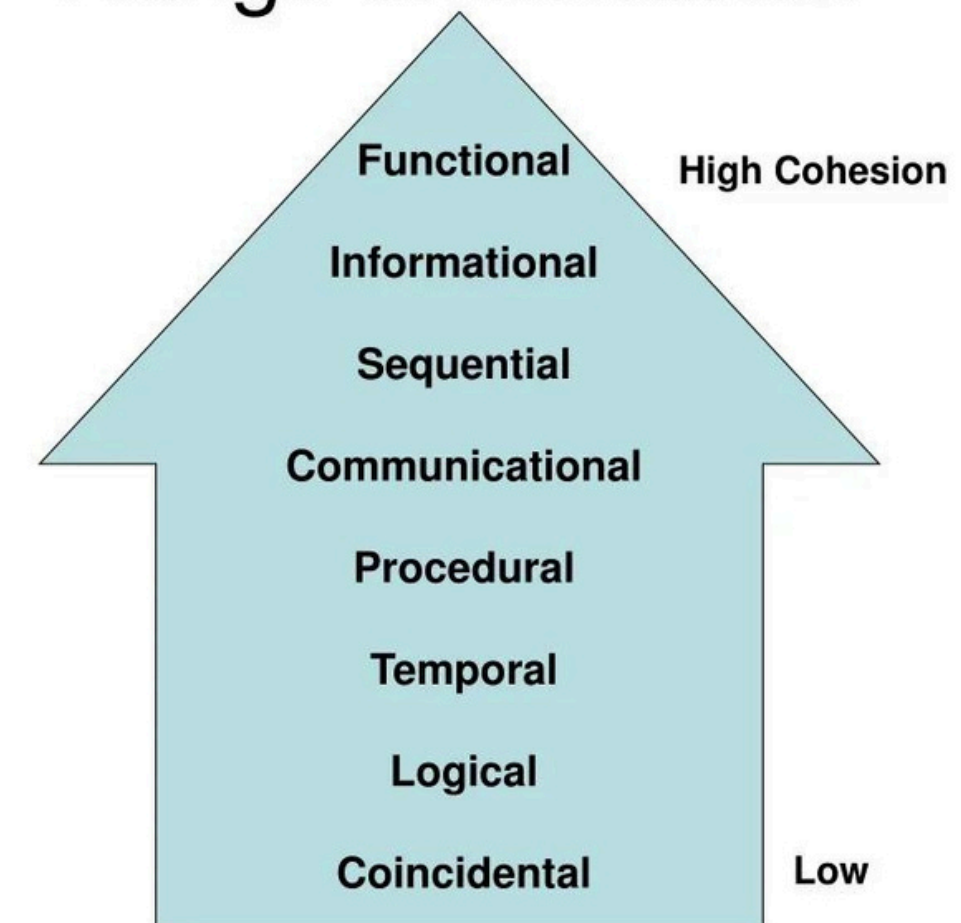
High Cohesion (ดี) Payment:

- calculateAmount()
- processPayment()
- generateReceipt()

Low Cohesion (ไม่ดี) Payment:

- processPayment()
- sendEmail()
- calculateSalary()

Range of Cohesion



หลักจำง่าย: Cohesion สูง = ภายในโมดูลมีหน้าที่ที่เกี่ยวข้องกันมาก

1.5 ระดับของ Cohesion (7 Levels of Cohesion)

เรียงลำดับจากระดับต่ำสุด (ไม่ดี) ไปสู่ระดับสูงสุด (ดีเยี่ยม):

1. Coincidental Cohesion (ต่ำสุด)

- นำงานมารวมกันโดยไม่มีความสัมพันธ์กันเลย

2. Logical Cohesion

- รวมงานประเภทเดียวกัน แต่เลือกทำงานต่างกันตามเงื่อนไข

3. Temporal Cohesion

- รวมงานที่เกิดขึ้นในช่วงเวลาเดียวกัน เช่น การเริ่มต้นระบบ (Initialization)

4. Procedural Cohesion

- งานต่าง ๆ ต้องทำตามลำดับขั้นตอนเดียวกัน

5. Communicational Cohesion

- งานต่าง ๆ ทำงานกับชุดข้อมูลเดียวกัน

6. Sequential Cohesion

- ผลลัพธ์จากงานหนึ่งถูกนำไปใช้เป็นข้อมูลของงานถัดไป

7. Functional Cohesion (สูงสุด/ดีที่สุด)

- ทุกส่วนร่วมกันทำหน้าที่เดียวที่ชัดเจน

❌ 1.6 Coupling (ความเชื่อมโยงระหว่างโมดูล)

๑ นิยาม Coupling

ระดับการพึ่งพาระหว่างคอมโพเนนต์หรือโมดูล
หนึ่งกับอีกโมดูลหนึ่ง

หลักการออกแบบที่ดี: Low Coupling (ความ
พึ่งพากันต่ำ)

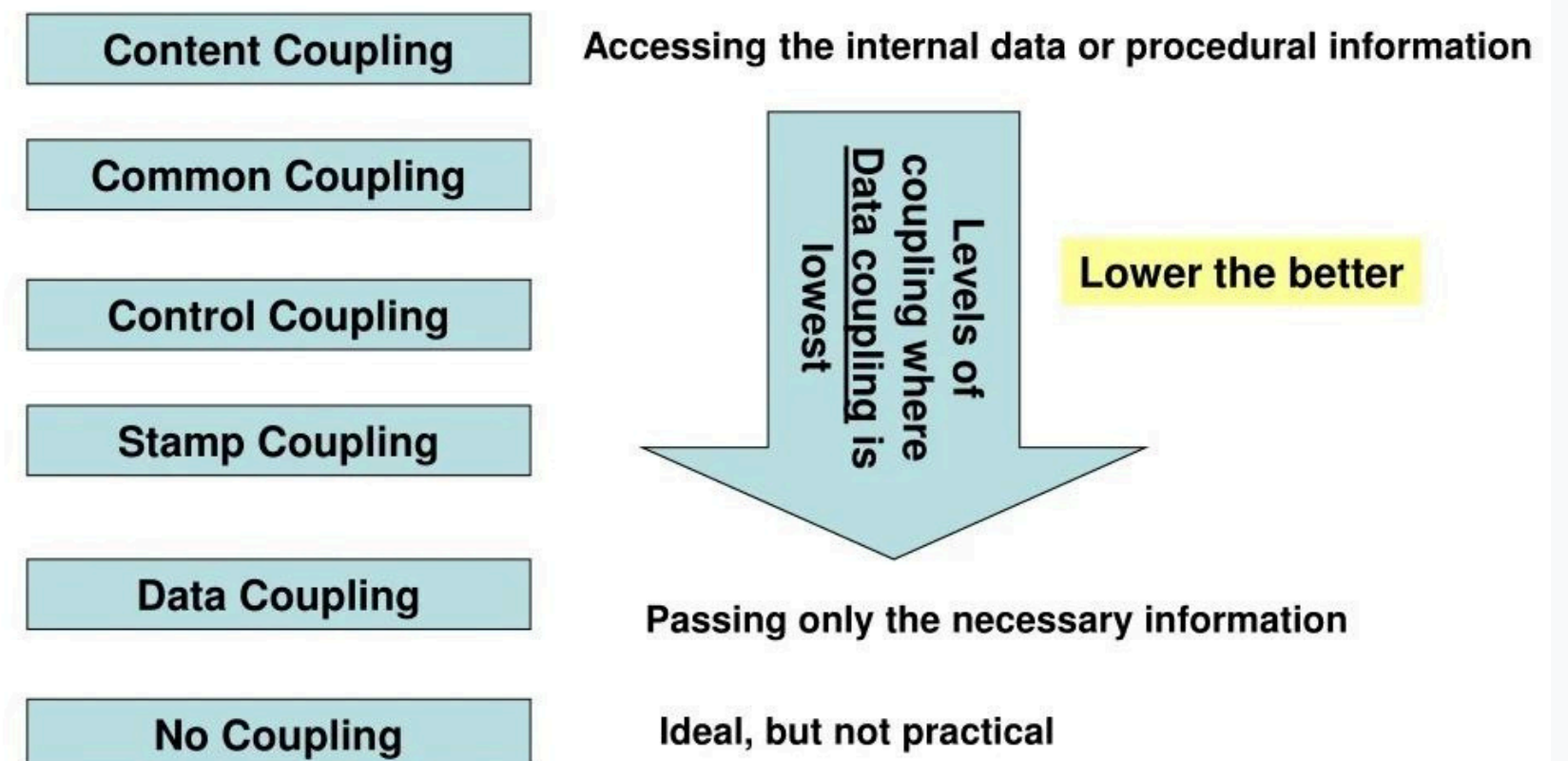
ตัวอย่าง Low Coupling:

Booking → Payment ส่งเฉพาะข้อมูลที่จำเป็น เช่น
booking_id และ amount ผ่าน Interface โดยไม่
ต้องรู้รายละเอียดภายในของ Payment

หลักจำง่าย: Coupling ต่ำ = โมดูลพึ่งพากันน้อย

Coupling

- Coupling addresses the attribute of “degree of interdependence” between software units, modules or components.



❏ 1.6 ประเภทของ Coupling (Levels of Coupling)

เรียงลำดับจากแนวโน้มที่ไม่ดีไปสู่แนวโน้มที่ดีที่สุด:

1. Content Coupling (แย่ที่สุด)

- โมดูลหนึ่งเข้าไปพึ่งพาหรือแก้ไขรายละเอียดภายในของอีกโมดูลโดยตรง

2. Common Coupling

- หลายโมดูลใช้ข้อมูลร่วมกัน เช่น Global Data

3. External Coupling

- พึ่งพาอินเทอร์เฟซหรือรูปแบบที่กำหนดโดยระบบภายนอก

4. Control Coupling

- โมดูลหนึ่งส่งข้อมูลควบคุมเพื่อกำหนดการทำงานของอีกโมดูล

5. Stamp Coupling

- ส่งโครงสร้างข้อมูลขนาดใหญ่ให้กัน ทั้งที่ใช้อีกโมดูลใช้เพียงบางส่วน

6. Data Coupling (ดีที่สุด)

- โมดูลสื่อสารกันโดยส่งเฉพาะข้อมูลที่จำเป็นเท่านั้น

✓ สรุปความสัมพันธ์ของ Cohesion และ Coupling

" High Cohesion + Low Coupling = การออกแบบที่ดี "

แนวคิด	หมายถึง	เป้าหมายการออกแบบ
Cohesion	ความเกี่ยวข้องกันองค์ประกอบภายในโมดูล	สูง (High) สูง (High) ต่ำ (Low)
Coupling	การพึ่งพากันและกันระหว่างโมดูล	ต่ำ (Low)

ทำไมถึงสำคัญ? เมื่อ Cohesion สูง แต่ละคอมโพเนนต์จะมีหน้าที่ชัดเจน ส่วนเมื่อ Coupling ต่ำ คอมโพเนนต์ต่าง ๆ จะไม่ผูกติดกันมากเกินไป ส่งผลให้ระบบสามารถแก้ไข ทดสอบ บำรุงรักษา และนำส่วนประกอบกลับมาใช้ซ้ำได้ง่ายขึ้น

เปรียบเทียบ Structured Design vs Object-Oriented Design

หัวข้อ	Structured Design	Object-Oriented Design (OOD)
สิ่งที่มองเป็นหลัก	ฟังก์ชัน / กระบวนการ	วัตถุ และ คลาส
การแบ่งระบบ	แบ่งเป็นโมดูลหรือฟังก์ชันย่อย	แบ่งเป็นคลาสและวัตถุ
ข้อมูล	มักแยกจากกระบวนการ	รวมข้อมูลกับพฤติกรรมเข้าด้วยกัน
แนวคิดสำคัญ	Decomposition, Data Flow	Encapsulation, Abstraction, Inheritance, Polymorphism
ความเหมาะสม	ระบบที่เน้นกระบวนการทำงาน	ระบบที่มีข้อมูลและพฤติกรรมซับซ้อน

สรุปสั้น ๆ:

- **Component:** ส่วนประกอบย่อยที่มีหน้าที่เฉพาะ | • **Component-Level Design:** การออกแบบรายละเอียดภายในและอินเทอร์เฟซ
- **Structured Design:** เน้นฟังก์ชัน/กระบวนการ | • **Object-Oriented Design:** เน้นวัตถุ คลาส ข้อมูลและพฤติกรรม
- **Cohesion:** ความสัมพันธ์ภายใน (ควรสูง) | • **Coupling:** ความพึ่งพภายนอก (ควรต่ำ)